

k-Armed Bandit Problem — Pedrito Cambolo

Reinforcement Learning | TecNM Ciudad Guzmán

Problem Description

This notebook implements the **k-armed bandit problem** applied to a serious educational math game called **Pedrito Cambolo**. In this game, an agent must decide which math operation to show the player at each step, with the goal of maximizing the number of correct answers over time.

The problem is modeled as follows:

- There are **10 possible actions** (math operations), ranging from easy additions to harder multiplications and unknown-variable problems.
- Each action has a **true probability of success** $q^*(a)$ — the likelihood that the player answers that operation correctly. These values are **permanently hidden from the agent**.
- At each step, the agent selects one operation, the player attempts to answer it, and the agent receives a reward of **1 (correct) or 0 (incorrect)**.
- The agent must learn, purely from experience, which operation gives the highest reward without ever knowing the true probabilities.

Agents Implemented

Agent	Policy	Initialization
Agent 1	Random — selects actions uniformly at random	—
Agent 2	Greedy — always exploits the highest Q-value	$Q_0 = 0$
Part 5	Greedy comparative	$Q_0 = 0$ / random / optimistic (5.0)

Learning Method

All agents estimate action values using the **sample average method**:

$$Q(a) = \frac{\sum \text{rewards from action } a}{\text{number of times } a \text{ was selected}}$$

Each experiment is evaluated over **500, 1000, 5000 and 10000 simulation steps**, averaged over **2000 independent runs** for the optimal action percentage plots.

Course: Aprendizaje por Refuerzo — Dr. Julio Alberto García Rodríguez

```
In [1]: # k-Armed Bandit Problem – Math Operations in a Serious Game
# Pedrito Cambolo: the agent selects which math operation to show the
# Reward = 1 if the player answers correctly, 0 if not
# The true probability of a correct answer is HIDDEN from the agent

import random
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "png"
random.seed(42)
np.random.seed(42)

In [2]: # — 10 math operations the game can show to the player —
# 4 types: suma, resta, multiplicacion, suma con incognita
# Each operation is one action for the k-armed bandit agent

operations = [
    "2 + 3 = ?",
    "14 + 8 = ?",
    "9 - 4 = ?",
    "17 - 9 = ?",
    "3 x 4 = ?",
    "6 x 7 = ?",
    "9 x 8 = ?",
    "? + 2 = 5",
    "? + 6 = 13",
    "? + 7 = 20"
]

# True probability that the player answers each operation correctly
# These are the real q*(a) values -- HIDDEN from the agent
real_probabilities = [0.90, 0.65, 0.85, 0.55, 0.75, 0.45, 0.30, 0.80,

K = len(operations)
optimal_action = int(np.argmax(real_probabilities)) # action with hig

print("Operation          | True q*(a)")
print("-" * 38)
```

```
for i, (op, p) in enumerate(zip(operations, real_probabilities)):
    marker = " <-- optimal" if i == optimal_action else ""
    print(f"{op:<22}| {p}{marker}")
```

Operation	True q*(a)
2 + 3 = ?	0.9 <-- optimal
14 + 8 = ?	0.65
9 - 4 = ?	0.85
17 - 9 = ?	0.55
3 x 4 = ?	0.75
6 x 7 = ?	0.45
9 x 8 = ?	0.3
? + 2 = 5	0.8
? + 6 = 13	0.6
? + 7 = 20	0.4

```
In [3]: # — Part 2: Stationary reward distribution for each operation —
# Each operation has a Bernoulli distribution: reward = 1 (correct) or
# We approximate it as a Gaussian N(q*(a), sigma) for visualization
# The agent can only observe samples -- it does NOT know these distrib

import plotly.graph_objects as go
from scipy.stats import norm

sigma = 0.10
x = np.linspace(0, 1, 400)

fig = go.Figure()

for i, mu in enumerate(real_probabilities):
    action_pos = i + 1
    y = norm.pdf(x, mu, sigma)
    y_norm = y / y.max() * 0.4

    fig.add_trace(go.Scatter(
        x=list(action_pos + y_norm) + list(action_pos - y_norm[::-1]),
        y=list(x) + list(x[::-1]),
        fill='toself',
        fillcolor='rgba(120,120,120,0.4)',
        line=dict(color='gray', width=0.5),
        name=f'a{i+1}: {operations[i]}',
        hovertemplate=f'<b>a{i+1}</b>: {operations[i]}<br>q*(a)={mu}<e
    ))

    fig.add_shape(type='line',
        x0=action_pos - 0.35, x1=action_pos + 0.35,
        y0=mu, y1=mu,
        line=dict(color='black', width=1.5)
    )

    fig.add_annotation(x=action_pos + 0.42, y=mu,
        text=f'q*({i+1})={mu}', showarrow=False, font=dict(size=9))
```

```
fig.add_hline(y=0.5, line_dash='dash', line_color='black', opacity=0.4)

fig.update_layout(
    title='Stationary Reward Distributions – 10 Math Operations<br>Pedrito Cambolo (Hidden from the agent)',
    xaxis=dict(tickvals=list(range(1, K+1)), title='Action'),
    yaxis=dict(title='Reward distribution', range=[-0.05, 1.1]),
    showlegend=True,
    width=950, height=500
)
fig.show()
```

```
from IPython.display import display, Markdown
display(Markdown("""
```

```
---
```

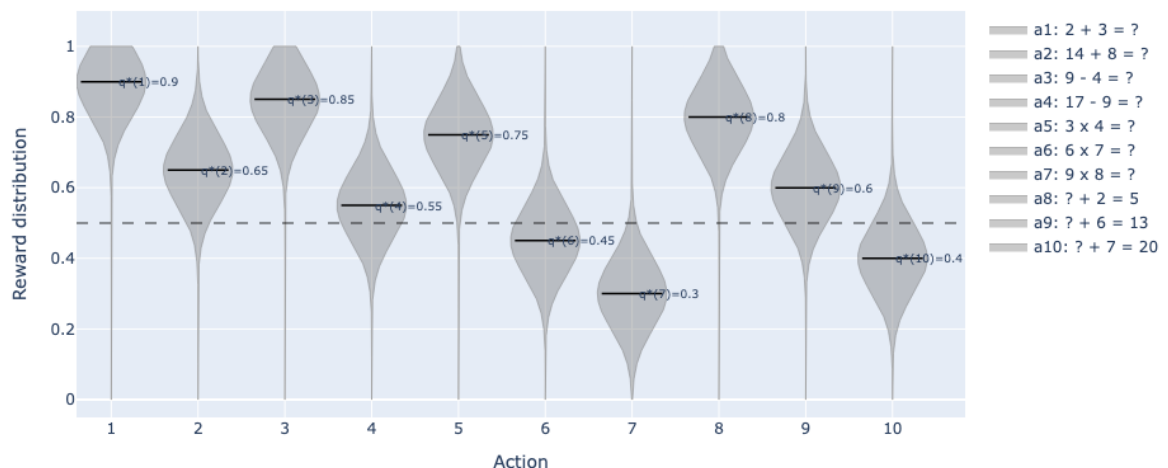
****Part 2 – Stationary Reward Distributions****

Each violin represents the reward distribution for one math operation. (1 = correct, 0 = wrong), each distribution is Bernoulli – approximate centered at $q^*(a)$ for visualization purposes.

- The **horizontal black line** inside each violin marks the true $q^*(a)$ long-run average reward the agent would receive if it always chose a .
- The **dashed horizontal line** at 0.5 serves as a reference midpoint.
- The **optimal action** is **a1: 2 + 3 = ?** with $q^* = 0.90$, meaning to answer correctly 90% of the time.
- The **hardest operation** is **a7: 9 x 8 = ?** with $q^* = 0.30$.

****Key point:**** The agent cannot observe these distributions – it can only learn by trying each operation and observing whether the player answered correctly or not.

Stationary Reward Distributions — 10 Math Operations
Pedrito Cambolo (Hidden from the agent)



Part 2 — Stationary Reward Distributions

Each violin represents the reward distribution for one math operation. Since rewards are binary (1 = correct, 0 = wrong), each distribution is Bernoulli — approximated here as a Gaussian centered at $q^*(a)$ for visualization purposes.

- The **horizontal black line** inside each violin marks the true $q^*(a)$ value, which is the long-run average reward the agent would receive if it always chose that action.
- The **dashed horizontal line** at 0.5 serves as a reference midpoint.
- The **optimal action** is **a1: $2 + 3 = ?$** with $q^* = 0.90$, meaning the player is expected to answer correctly 90% of the time.
- The **hardest operation** is **a7: $9 \times 8 = ?$** with $q^* = 0.30$.

Key point: The agent cannot observe these distributions — it can only collect samples by trying each operation and observing whether the player answered correctly or not.

```
In [4]: # — Operation class —————
# Represents one math operation the game can show to the player

class MathOperation:
    def __init__(self, name, real_prob):
        self.name          = name
        self.real_prob      = real_prob    # hidden from the agent

        self.q_value       = 0.0
        self.count          = 0
        self.accum_reward   = 0.0

    def show_to_player(self):
        # Simulate the player's response: 1 = correct, 0 = wrong
        return 1 if random.random() < self.real_prob else 0

    def learn(self, reward):
        # Update Q-value using sample average (incremental update)
        self.count          += 1
        self.accum_reward   += reward
        self.q_value        = self.accum_reward / self.count

    def __str__(self):
        return (f"{self.name:<22} -> Q: {self.q_value:.4f} "
                f"(Shown {self.count} times)")
```

```
In [5]: # — Part 3: Agent 1 – Random selection policy (pure exploration) —————
# The agent selects an operation completely at random each step
```

```

# It still records Q-values via sample average -- but never uses them
# n_runs = 2000 independent runs for the optimal-action percentage plot

def run_random_agent(n_steps):
    ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]

    tried_history = [] # operation name chosen at each step
    q_history = [] # Q-values snapshot at each step
    accum_history = [] # accumulated rewards snapshot
    cumreward_curve = [] # running total reward
    total_reward = 0.0

    for t in range(n_steps):
        selected = random.choice(ops) # random policy
        reward = selected.show_to_player() # environment response
        selected.learn(reward)

        total_reward += reward
        tried_history.append(selected.name)
        q_history.append([op.q_value for op in ops])
        accum_history.append([op.accum_reward for op in ops])
        cumreward_curve.append(total_reward)

    return ops, tried_history, np.array(q_history), np.array(accum_history), cumreward_curve

def compute_optimal_pct(n_steps, n_runs=2000):
    # Matrix: rows = runs, cols = steps. 1 if optimal operation was chosen
    optimal_matrix = np.zeros((n_runs, n_steps))

    for run in range(n_runs):
        ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]
        for t in range(n_steps):
            selected = random.choice(ops)
            reward = selected.show_to_player()
            selected.learn(reward)
            if selected.name == operations[optimal_action]:
                optimal_matrix[run, t] = 1

    return np.mean(optimal_matrix, axis=0) * 100

print("Random agent and optimal-percentage functions ready.")

```

Random agent and optimal-percentage functions ready.

```

In [6]: # — Plots: 4 subplots for each of the 4 simulation lengths —
# (a) Times each operation was tried
# (b) Q-value estimates over time
# (c) Accumulated reward over time
# (d) % optimal operation selected – averaged over 2000 runs

import plotly.graph_objects as go

```

```

from plotly.subplots import make_subplots

colors_plotly = [
    '#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
    '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf'
]

STEPS_LIST = [500, 1000, 5000, 10000]

for n_steps in STEPS_LIST:

    ops, tried_hist, q_arr, accum_arr, cumrew = run_random_agent(n_steps)
    pct_opt = compute_optimal_pct(n_steps)
    steps = list(range(1, n_steps + 1))
    counts = [tried_hist.count(op) for op in operations]

    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=[
            '(a) Times each operation was tried',
            '(b) Action-value estimates over time',
            '(c) Accumulated reward over time',
            '(d) % Optimal action selected (2000 runs)'
        ]
    )

    # (a) barras de conteo
    for i, (count, op) in enumerate(zip(counts, operations)):
        fig.add_trace(go.Bar(
            x=[f'a{i+1}'],
            y=[count],
            name=op,
            marker_color=colors_plotly[i],
            marker_line_color='red' if i == optimal_action else 'black',
            marker_line_width=3 if i == optimal_action else 0.5,
            hovertemplate=f'<b>{op}</b><br>Shown: {count} times<extra>
            showlegend=False
        ), row=1, col=1)

    # (b) Q-values por operacion
    for i, op in enumerate(operations):
        fig.add_trace(go.Scatter(
            x=steps, y=q_arr[:, i],
            name=op,
            line=dict(color=colors_plotly[i], width=1),
            hovertemplate=f'<b>{op}</b><br>Step: %{{x}}<br>Q: %{{y:.4f}}
        ), row=1, col=2)
        fig.add_hline(y=real_probabilities[i],
            line_dash='dot',
            line_color=colors_plotly[i],
            opacity=0.4, row=1, col=2)

```

```

# (c) recompensa acumulada
fig.add_trace(go.Scatter(
    x=steps, y=cumrew,
    name='Accumulated reward',
    line=dict(color='steelblue', width=1.5),
    hovertemplate='Step: %{x}<br>Total correct: %{y:.0f}<extra></extra>',
    showlegend=False
), row=2, col=1)

# (d) % accion optima
fig.add_trace(go.Scatter(
    x=steps, y=pct_opt,
    name='Random agent',
    line=dict(color='darkorange', width=1.2),
    hovertemplate='Step: %{x}<br>% Optimal: %{y:.2f}%<extra></extra>',
    showlegend=False
), row=2, col=2)
fig.add_hline(y=100/K, line_dash='dash', line_color='gray',
              annotation_text=f'Chance ({100/K:.1f}%)', row=2, col=2)

fig.update_layout(
    title=f'Agent 1 – Random Policy | Pedrito Cambolo | Steps = {n}',
    height=750, width=1100,
    hovermode='x unified'
)
fig.update_yaxes(range=[0, 40], row=2, col=2)
fig.show()

from IPython.display import display, Markdown
explanations = {
    500: """
---
**Agent 1 – Random Policy | Steps = 500**

**(a)** With only 500 steps and random selection, all 10 operations are equally likely. No operation is preferred – the agent has no mechanism to favor any action.

**(b)** Q-value estimates are noisy and far from the true q*(a) values. With so few samples per action, the estimates have not converged yet.

**(c)** Accumulated reward grows linearly at a constant rate, reflecting the fact that across all operations (~0.625 expected correct per step).

**(d)** The % optimal action hovers around ~10% – exactly what is expected with 10 equally likely actions. The random agent never learns to prefer any action.
""",
    1000: """
---
**Agent 1 – Random Policy | Steps = 1000**

**(a)** Action counts remain approximately uniform (~100 times each). The agent continues to distribute attempts evenly regardless of past rewards.
"""
}

```



```

**(b)** Q-value estimates improve slightly compared to 500 steps but r
The optimal action (a1) estimate starts to approach 0.90 but is still

**(c)** Accumulated reward continues its linear growth. No improvement
is observed because the agent does not exploit what it has learned.

**(d)** % optimal action still oscillates around 10%. Doubling the ste
because the policy is random – more experience provides no benefit in
"""
    5000: """
---
**Agent 1 – Random Policy | Steps = 5000**

**(a)** With 5000 steps, each operation has been shown approximately 5
The distribution is still uniform – the random agent is completely ind

**(b)** Q-value estimates are now much closer to the true q*(a) values
action, the sample averages have converged. However, this information

**(c)** Accumulated reward grows steadily. The slope reflects the aver
actions (~0.625), unchanged from previous step counts.

**(d)** % optimal action remains flat near 10%. This confirms that the
incapable of improving its behavior regardless of how much experience
"""
    10000: """
---
**Agent 1 – Random Policy | Steps = 10000**

**(a)** At 10,000 steps each operation has been tried ~1,000 times. Pe
across all actions, as expected from a random policy.

**(b)** Q-value estimates have now converged very close to their true
The agent has an accurate internal model of the environment – but neve

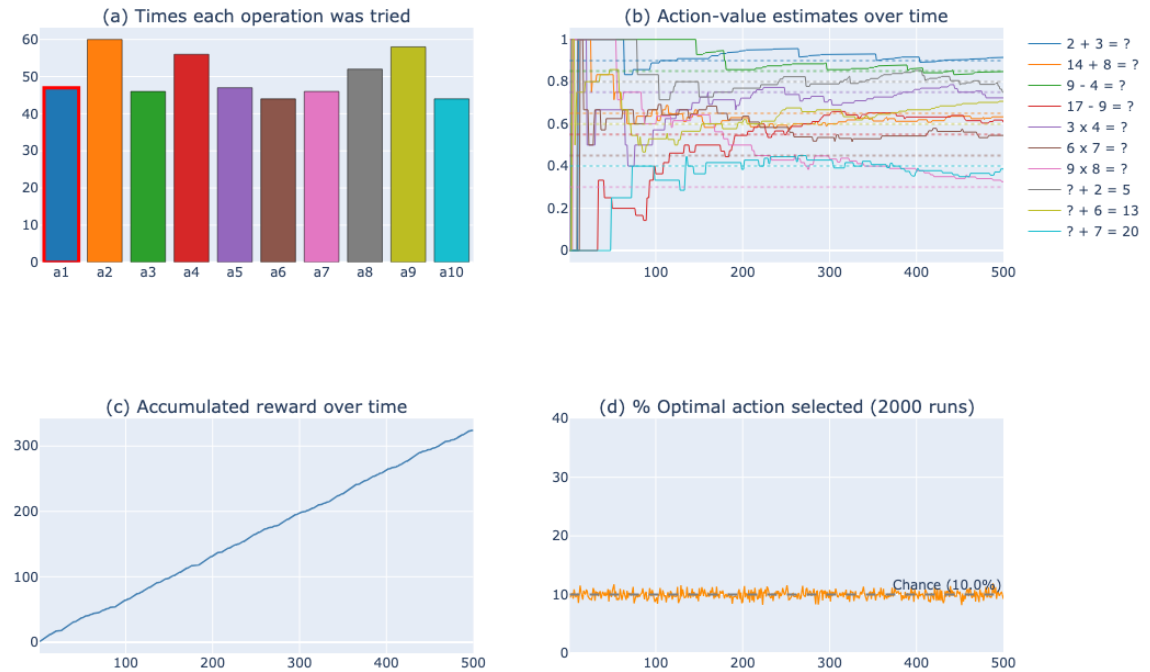
**(c)** The accumulated reward curve is the clearest linear trend of a
Total correct answers  $\approx 6,250$  ( $10,000 \times 0.625$  average probability).

**(d)** % optimal action is stable at ~10%. This is the theoretical ba
agent with K=10 actions. **Conclusion:** a random policy is the worst
it demonstrates why we need agents that exploit their learned knowledg
"""
    }
    display(Markdown(explanations[n_steps]))

    # Final Q-values -- same as Doctor's final print
    print(f"\n--- Final Q-values (n_steps={n_steps}) ---")
    for op in ops:
        print(op)

```

Agent 1 — Random Policy | Pedrito Cambolo | Steps = 500



Agent 1 — Random Policy | Steps = 500

(a) With only 500 steps and random selection, all 10 operations are tried roughly equally (~50 times each). No operation is preferred — the agent has no mechanism to favor any action.

(b) Q-value estimates are noisy and far from the true $q^*(a)$ values (dotted lines). With so few samples per action, the estimates have not converged yet.

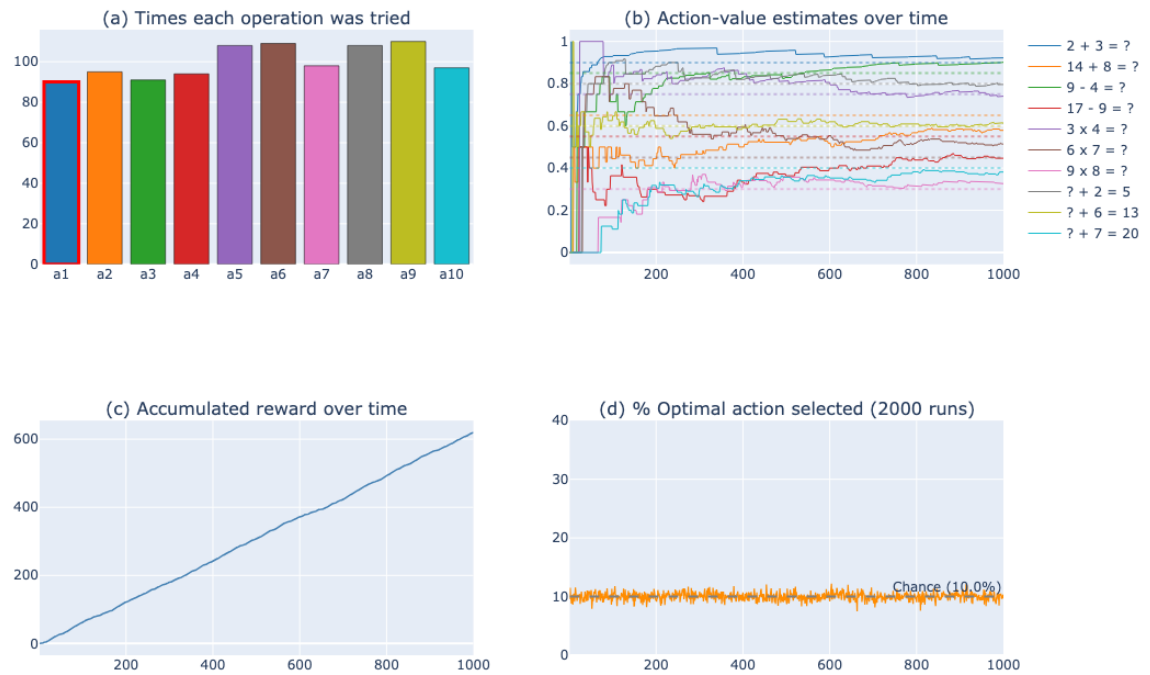
(c) Accumulated reward grows linearly at a constant rate, reflecting the average probability across all operations (~0.625 expected correct per step).

(d) The % optimal action hovers around **10%** — exactly what is expected by pure chance with 10 equally likely actions. The random agent never learns to prefer the best operation.

--- Final Q-values (n_steps=500) ---

$2 + 3 = ?$	-> Q: 0.9149	(Shown 47 times)
$14 + 8 = ?$	-> Q: 0.6333	(Shown 60 times)
$9 - 4 = ?$	-> Q: 0.8478	(Shown 46 times)
$17 - 9 = ?$	-> Q: 0.6071	(Shown 56 times)
$3 \times 4 = ?$	-> Q: 0.7234	(Shown 47 times)
$6 \times 7 = ?$	-> Q: 0.5455	(Shown 44 times)
$9 \times 8 = ?$	-> Q: 0.3261	(Shown 46 times)
$? + 2 = 5$	-> Q: 0.7500	(Shown 52 times)
$? + 6 = 13$	-> Q: 0.7069	(Shown 58 times)
$? + 7 = 20$	-> Q: 0.3864	(Shown 44 times)

Agent 1 — Random Policy | Pedrito Cambolo | Steps = 1000



Agent 1 — Random Policy | Steps = 1000

(a) Action counts remain approximately uniform (~100 times each). The random policy continues to distribute attempts evenly regardless of past rewards.

(b) Q-value estimates improve slightly compared to 500 steps but remain noisy. The optimal action (a1) estimate starts to approach 0.90 but is still unstable.

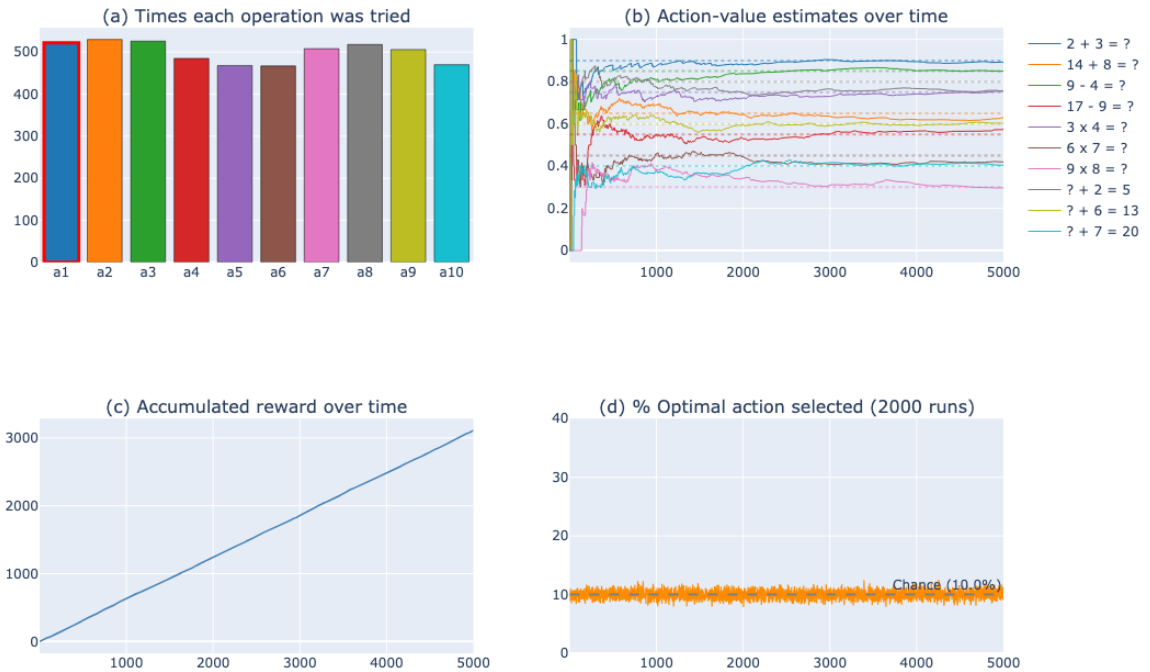
(c) Accumulated reward continues its linear growth. No improvement over the random baseline is observed because the agent does not exploit what it has learned.

(d) % optimal action still oscillates around 10%. Doubling the steps made no difference because the policy is random — more experience provides no benefit in decision-making.

--- Final Q-values (n_steps=1000) ---

2 + 3 = ?	-> Q: 0.9222	(Shown 90 times)
14 + 8 = ?	-> Q: 0.5789	(Shown 95 times)
9 - 4 = ?	-> Q: 0.9011	(Shown 91 times)
17 - 9 = ?	-> Q: 0.4468	(Shown 94 times)
3 x 4 = ?	-> Q: 0.7407	(Shown 108 times)
6 x 7 = ?	-> Q: 0.5138	(Shown 109 times)
9 x 8 = ?	-> Q: 0.3265	(Shown 98 times)
? + 2 = 5	-> Q: 0.7963	(Shown 108 times)
? + 6 = 13	-> Q: 0.6091	(Shown 110 times)
? + 7 = 20	-> Q: 0.3814	(Shown 97 times)

Agent 1 — Random Policy | Pedrito Cambolo | Steps = 5000



Agent 1 — Random Policy | Steps = 5000

(a) With 5000 steps, each operation has been shown approximately 500 times. The distribution is still uniform — the random agent is completely indifferent to outcomes.

(b) Q-value estimates are now much closer to the true $q^*(a)$ values. With ~500 samples per action, the sample averages have converged. However, this information is never used to decide.

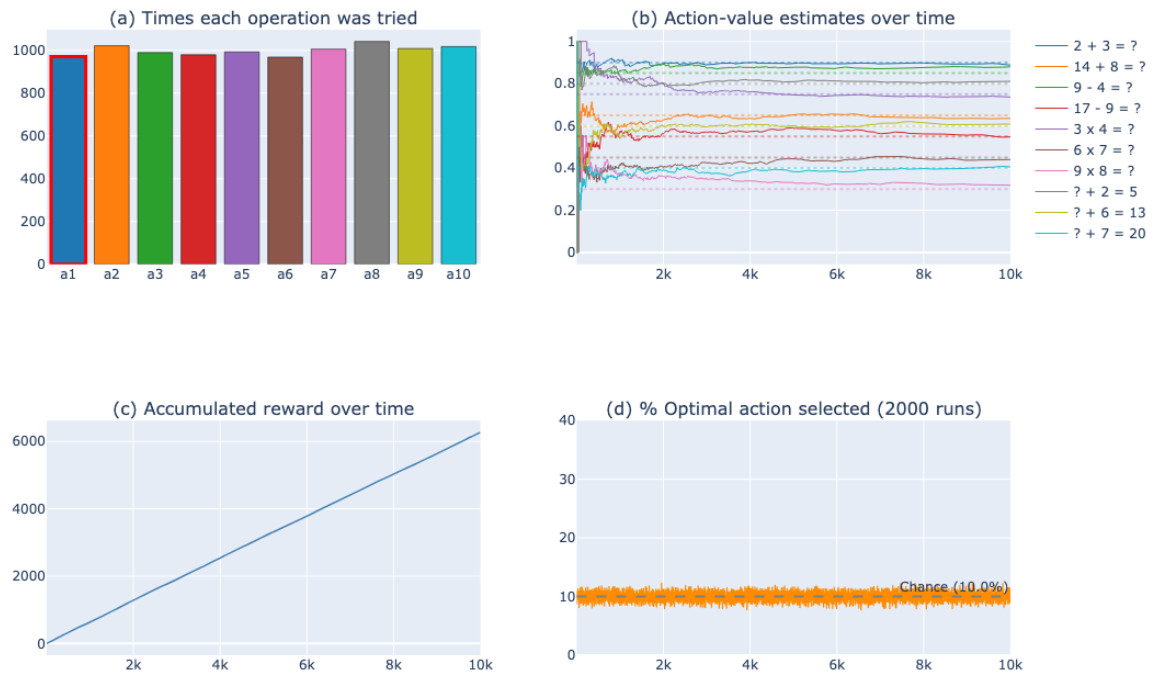
(c) Accumulated reward grows steadily. The slope reflects the average reward across all actions (~ 0.625), unchanged from previous step counts.

(d) % optimal action remains flat near 10%. This confirms that the random agent is incapable of improving its behavior regardless of how much experience it accumulates.

--- Final Q-values (n_steps=5000) ---

2 + 3 = ?	-> Q: 0.8927	(Shown 522 times)
14 + 8 = ?	-> Q: 0.6302	(Shown 530 times)
9 - 4 = ?	-> Q: 0.8498	(Shown 526 times)
17 - 9 = ?	-> Q: 0.5732	(Shown 485 times)
3 x 4 = ?	-> Q: 0.7564	(Shown 468 times)
6 x 7 = ?	-> Q: 0.4197	(Shown 467 times)
9 x 8 = ?	-> Q: 0.2972	(Shown 508 times)
? + 2 = 5	-> Q: 0.7548	(Shown 518 times)
? + 6 = 13	-> Q: 0.6028	(Shown 506 times)
? + 7 = 20	-> Q: 0.4043	(Shown 470 times)

Agent 1 — Random Policy | Pedrito Cambolo | Steps = 10000



Agent 1 — Random Policy | Steps = 10000

(a) At 10,000 steps each operation has been tried ~1,000 times. Perfectly uniform selection across all actions, as expected from a random policy.

(b) Q-value estimates have now converged very close to their true $q^*(a)$ values. The agent has an accurate internal model of the environment — but never uses it.

(c) The accumulated reward curve is the clearest linear trend of all step counts. Total correct answers $\approx 6,250$ ($10,000 \times 0.625$ average probability).

(d) % optimal action is stable at ~10%. This is the theoretical baseline for a random agent with $K=10$ actions. **Conclusion:** a random policy is the worst possible strategy — it demonstrates why we need agents that exploit their learned knowledge.

```

--- Final Q-values (n_steps=10000) ---
2 + 3 = ?      -> Q: 0.8888 (Shown 971 times)
14 + 8 = ?     -> Q: 0.6350 (Shown 1022 times)
9 - 4 = ?      -> Q: 0.8798 (Shown 990 times)
17 - 9 = ?     -> Q: 0.5480 (Shown 980 times)
3 x 4 = ?      -> Q: 0.7351 (Shown 993 times)
6 x 7 = ?      -> Q: 0.4407 (Shown 969 times)
9 x 8 = ?      -> Q: 0.3191 (Shown 1006 times)
? + 2 = 5      -> Q: 0.8109 (Shown 1042 times)
? + 6 = 13     -> Q: 0.6075 (Shown 1009 times)
? + 7 = 20     -> Q: 0.4057 (Shown 1018 times)

```

```

In [7]: # Part 4: Agent 2 – Greedy Policy (exploitation only)
        # Always selects the action with the highest Q-value using argmax()
        # Ties broken randomly. Q-values initialized to zero.

def run_greedy_agent(n_steps):
    ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]

    for op in ops:
        op.q_value = 0.0

    tried_history = []
    q_history = []
    accum_history = []
    cumreward_curve = []
    total_reward = 0.0

    for t in range(n_steps):
        max_q = max(op.q_value for op in ops)
        best_ops = [op for op in ops if op.q_value == max_q] # collect all best ops
        selected = random.choice(best_ops) # break ties randomly

        reward = selected.show_to_player()
        selected.learn(reward)

        total_reward += reward
        tried_history.append(selected.name)
        q_history.append([op.q_value for op in ops])
        accum_history.append([op.accum_reward for op in ops])
        cumreward_curve.append(total_reward)

    return ops, tried_history, np.array(q_history), np.array(accum_history), cumreward_curve

def compute_optimal_pct_greedy(n_steps, n_runs=2000):
    optimal_matrix = np.zeros((n_runs, n_steps)) # rows = runs, cols = steps

    for run in range(n_runs):
        ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]
        for op in ops:
            op.q_value = 0.0 # reset for each run

```

```

    for t in range(n_steps):
        max_q = max(op.q_value for op in ops)
        best_ops = [op for op in ops if op.q_value == max_q]
        selected = random.choice(best_ops)
        reward = selected.show_to_player()
        selected.learn(reward)
        if selected.name == operations[optimal_action]:
            optimal_matrix[run, t] = 1

    return np.mean(optimal_matrix, axis=0) * 100

print("Greedy agent functions ready.")

```

Greedy agent functions ready.

In [8]: *# Plots: 4 subplots for each simulation length – Greedy agent (Q0 = 0)*

```

import plotly.graph_objects as go
from plotly.subplots import make_subplots

colors_plotly = [
    '#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
    '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf'
]

STEPS_LIST = [500, 1000, 5000, 10000]

for n_steps in STEPS_LIST:

    ops, tried_hist, q_arr, accum_arr, cumrew = run_greedy_agent(n_steps)
    pct_opt = compute_optimal_pct_greedy(n_steps)
    steps = list(range(1, n_steps + 1))
    counts = [tried_hist.count(op) for op in operations]

    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=[
            '(a) Times each operation was tried',
            '(b) Action-value estimates over time',
            '(c) Accumulated reward over time',
            '(d) % Optimal action selected (2000 runs)'
        ]
    )

    # (a) action counts
    for i, (count, op) in enumerate(zip(counts, operations)):
        fig.add_trace(go.Bar(
            x=[f'a{i+1}'], y=[count], name=op,
            marker_color=colors_plotly[i],
            marker_line_color='red' if i == optimal_action else 'black',
            marker_line_width=3 if i == optimal_action else 0.5,
            hovertemplate=f'<b>{op}</b><br>Shown: {count} times<extra>

```



```

        showlegend=False
    ), row=1, col=1)

# (b) Q-value estimates – dotted lines = true q* (hidden from agent)
for i, op in enumerate(operations):
    fig.add_trace(go.Scatter(
        x=steps, y=q_arr[:, i], name=op,
        line=dict(color=colors_plotly[i], width=1),
        hovertemplate=f'<b>{op}</b><br>Step: %{{x}}<br>Q: %{{y:.4f}}',
    ), row=1, col=2)
    fig.add_hline(y=real_probabilities[i], line_dash='dot',
        line_color=colors_plotly[i], opacity=0.4, row=1,

# (c) accumulated reward
fig.add_trace(go.Scatter(
    x=steps, y=cumrew, name='Accumulated reward',
    line=dict(color='green', width=1.5),
    hovertemplate='Step: %{{x}}<br>Total correct: %{{y:.0f}}<extra></extra>',
    showlegend=False
), row=2, col=1)

# (d) % optimal action over 2000 runs
fig.add_trace(go.Scatter(
    x=steps, y=pct_opt, name='Greedy agent',
    line=dict(color='crimson', width=1.2),
    hovertemplate='Step: %{{x}}<br>% Optimal: %{{y:.2f}}%<extra></extra>',
    showlegend=False
), row=2, col=2)
fig.add_hline(y=100/K, line_dash='dash', line_color='gray',
    annotation_text=f'Chance ({100/K:.1f}%)', row=2, col=2)

fig.update_layout(
    title=f'Agent 2 – Greedy Policy (Q0=0) | Pedrito Cambolo | Steps = 500',
    height=750, width=1100,
    hovermode='x unified'
)
fig.update_yaxes(range=[0, 100], row=2, col=2)
fig.show()

from IPython.display import display, Markdown
explanations = {
    500: """
---
**Agent 2 – Greedy Policy (Q0=0) | Steps = 500**

** (a) ** Unlike the random agent, the greedy agent concentrates its attention on a single action. Once an action produces a reward, its Q-value rises above 0 and becomes dominant. This creates a strong imbalance – most actions are tried only once or twice.

** (b) ** The Q-value of the selected dominant action converges quickly. However, the agent may have locked onto a suboptimal action early due to lucky chance.
    """
}

```

```

**(c)** Accumulated reward grows faster than the random agent if the g
But if it got stuck on a suboptimal one, performance may be comparable

**(d)** % optimal action can be significantly above 10% if the agent h
but may also be low if it committed early to a different operation. Hi
"""
    1000: """
---
**Agent 2 – Greedy Policy ( $Q_0=0$ ) | Steps = 1000**

**(a)** The concentration effect is even more pronounced. One or two a
while most others were barely explored. The agent commits early and ex

**(b)** The dominant action's Q-value has stabilized. The estimates of
remain near 0 or their single-sample value – they were never given a c

**(c)** If the dominant action is near the optimal, accumulated reward
The greedy agent's strength is fast convergence when it finds a good a

**(d)** % optimal action is more stable than at 500 steps. The agent h
action and the 2000-run average shows whether that commitment was corr
"""
    5000: """
---
**Agent 2 – Greedy Policy ( $Q_0=0$ ) | Steps = 5000**

**(a)** By 5000 steps the action selection is almost entirely concentr
The greedy policy has fully committed – this is pure exploitation with

**(b)** The Q-value of the chosen action converges tightly to its true
actions stay near their initial estimate of 0 since they were rarely r

**(c)** Accumulated reward curve shows a steeper slope than the random
action is high-quality. The greedy agent's long-term performance depen

**(d)** % optimal action reveals the greedy agent's weakness: it may p
it committed to a suboptimal action in many of the 2000 runs and never
"""
    10000: """
---
**Agent 2 – Greedy Policy ( $Q_0=0$ ) | Steps = 10000**

**(a)** At 10,000 steps the action count is extremely concentrated. Th
nearly all steps on its chosen dominant action, leaving the rest almos

**(b)** The estimated Q-value of the dominant action is very accurate.
remain unreliable due to lack of samples – this is the cost of pure ex

**(c)** Total accumulated reward is high if the dominant action is opt
curve's slope reveals whether the agent found the best action or settl

**(d)** **Key takeaway:** even at 10,000 steps, the greedy agent's % o

```

significantly lower than 100%. This demonstrates the exploration-exploitation trade-off. A policy that never explores risks permanently missing the true best action.

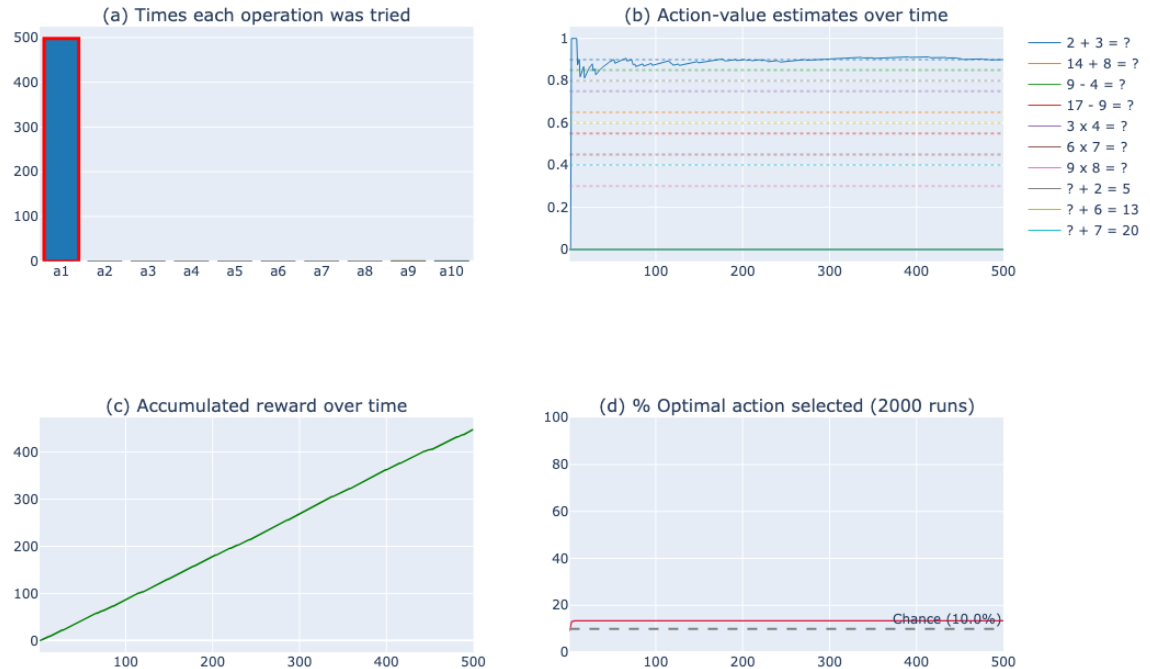
```

}
display(Markdown(explanations[n_steps]))

print(f"\n--- Final Q-values Greedy (n_steps={n_steps}) ---")
for op in ops:
    print(op)

```

Agent 2 — Greedy Policy (Q0=0) | Pedrito Cambolo | Steps = 500



Agent 2 — Greedy Policy ($Q_0=0$) | Steps = 500

(a) Unlike the random agent, the greedy agent concentrates its attempts on very few operations. Once an action produces a reward, its Q-value rises above 0 and becomes the preferred choice. This creates a strong imbalance — most actions are tried only once or twice.

(b) The Q-value of the selected dominant action converges quickly. However, with only 500 steps, the agent may have locked onto a suboptimal action early due to lucky initial rewards.

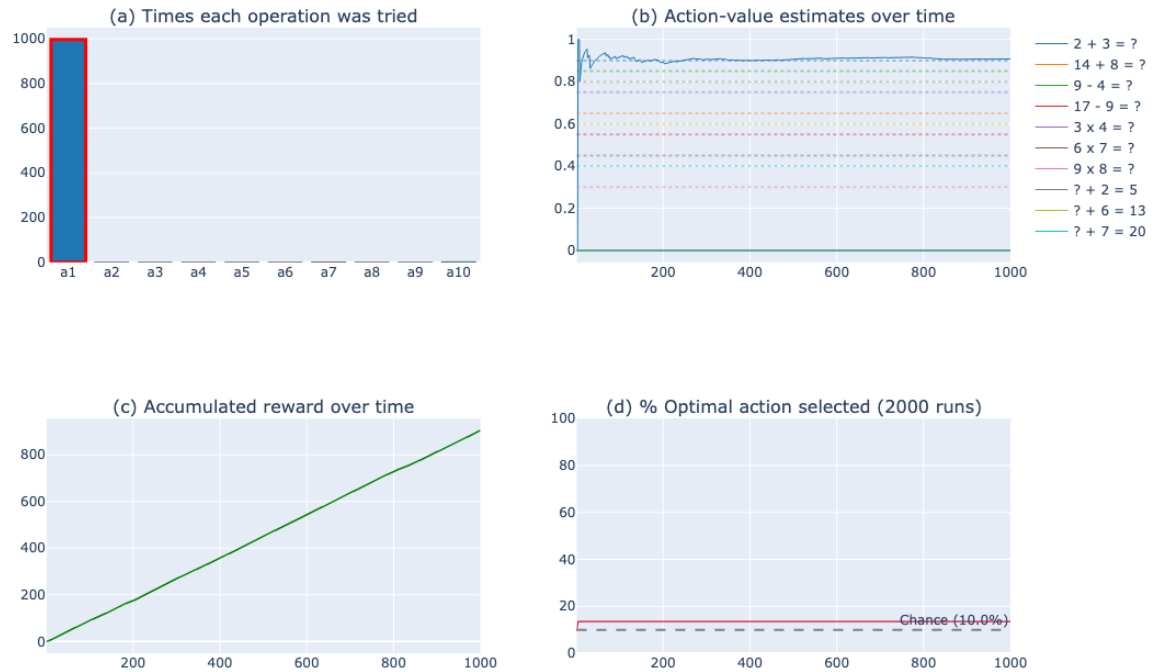
(c) Accumulated reward grows faster than the random agent if the greedy agent found a good action. But if it got stuck on a suboptimal one, performance may be comparable or worse.

(d) % optimal action can be significantly above 10% if the agent happened to try a1 first, but may also be low if it committed early to a different operation. High variance at this step count.

```

--- Final Q-values Greedy (n_steps=500) ---
2 + 3 = ?      -> Q: 0.8996 (Shown 498 times)
14 + 8 = ?     -> Q: 0.0000 (Shown 0 times)
9 - 4 = ?      -> Q: 0.0000 (Shown 0 times)
17 - 9 = ?     -> Q: 0.0000 (Shown 0 times)
3 x 4 = ?      -> Q: 0.0000 (Shown 0 times)
6 x 7 = ?      -> Q: 0.0000 (Shown 0 times)
9 x 8 = ?      -> Q: 0.0000 (Shown 0 times)
? + 2 = 5      -> Q: 0.0000 (Shown 0 times)
? + 6 = 13     -> Q: 0.0000 (Shown 1 times)
? + 7 = 20     -> Q: 0.0000 (Shown 1 times)

```

Agent 2 — Greedy Policy ($Q_0=0$) | Pedrito Cambolo | Steps = 1000Agent 2 — Greedy Policy ($Q_0=0$) | Steps = 1000

(a) The concentration effect is even more pronounced. One or two actions dominate the count while most others were barely explored. The agent commits early and exploits aggressively.

(b) The dominant action's Q-value has stabilized. The estimates of rarely-chosen actions remain near 0 or their single-sample value — they were never given a chance to converge.

(c) If the dominant action is near the optimal, accumulated reward outpaces the random agent. The greedy agent's strength is fast convergence when it finds a good action early.

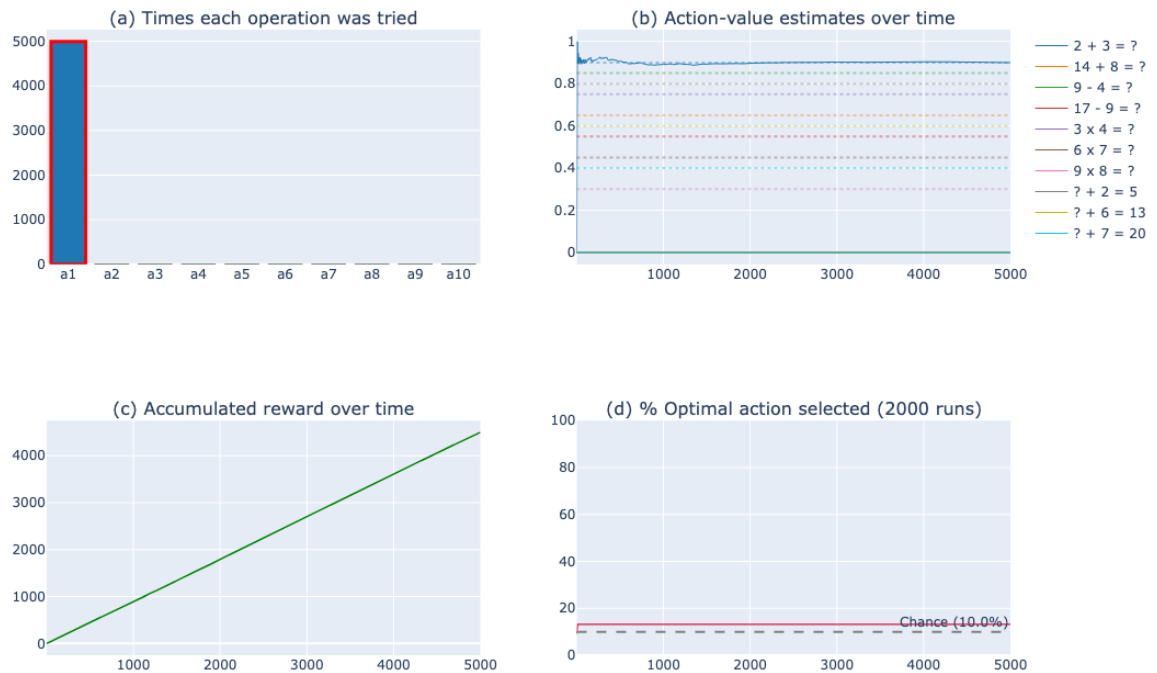
(d) % optimal action is more stable than at 500 steps. The agent has committed to its chosen action and the 2000-run average shows whether that commitment was correct across runs.

```

--- Final Q-values Greedy (n_steps=1000) ---
2 + 3 = ?          -> Q: 0.9077 (Shown 997 times)
14 + 8 = ?         -> Q: 0.0000 (Shown 0 times)
9 - 4 = ?          -> Q: 0.0000 (Shown 0 times)
17 - 9 = ?         -> Q: 0.0000 (Shown 0 times)
3 x 4 = ?          -> Q: 0.0000 (Shown 0 times)
6 x 7 = ?          -> Q: 0.0000 (Shown 0 times)
9 x 8 = ?          -> Q: 0.0000 (Shown 1 times)
? + 2 = 5          -> Q: 0.0000 (Shown 0 times)
? + 6 = 13         -> Q: 0.0000 (Shown 0 times)
? + 7 = 20         -> Q: 0.0000 (Shown 2 times)

```

Agent 2 — Greedy Policy (Q0=0) | Pedrito Cambolo | Steps = 5000



Agent 2 — Greedy Policy ($Q_0=0$) | Steps = 5000

(a) By 5000 steps the action selection is almost entirely concentrated on 1–2 operations. The greedy policy has fully committed — this is pure exploitation with minimal exploration.

(b) The Q-value of the chosen action converges tightly to its true $q^*(a)$. The remaining actions stay near their initial estimate of 0 since they were rarely revisited.

(c) Accumulated reward curve shows a steeper slope than the random agent if the committed action is high-quality. The greedy agent's long-term performance depends heavily on its early choices.

(d) % optimal action reveals the greedy agent's weakness: it may plateau below 100% because it committed to a suboptimal action in many of the 2000 runs and never recovered.

```

--- Final Q-values Greedy (n_steps=5000) ---
2 + 3 = ?          -> Q: 0.9000 (Shown 4999 times)
14 + 8 = ?         -> Q: 0.0000 (Shown 0 times)
9 - 4 = ?          -> Q: 0.0000 (Shown 0 times)
17 - 9 = ?         -> Q: 0.0000 (Shown 0 times)
3 x 4 = ?          -> Q: 0.0000 (Shown 0 times)
6 x 7 = ?          -> Q: 0.0000 (Shown 0 times)
9 x 8 = ?          -> Q: 0.0000 (Shown 0 times)
? + 2 = 5          -> Q: 0.0000 (Shown 0 times)
? + 6 = 13         -> Q: 0.0000 (Shown 0 times)
? + 7 = 20         -> Q: 0.0000 (Shown 1 times)

```

Agent 2 — Greedy Policy ($Q_0=0$) | Pedrito Cambolo | Steps = 10000Agent 2 — Greedy Policy ($Q_0=0$) | Steps = 10000

(a) At 10,000 steps the action count is extremely concentrated. The greedy agent has spent nearly all steps on its chosen dominant action, leaving the rest almost unexplored.

(b) The estimated Q-value of the dominant action is very accurate. All other Q-values remain unreliable due to lack of samples — this is the cost of pure exploitation.

(c) Total accumulated reward is high if the dominant action is optimal, but the reward curve's slope reveals whether the agent found the best action or settled for a good-enough one.

(d) Key takeaway: even at 10,000 steps, the greedy agent's % optimal action may be significantly lower than 100%. This demonstrates the exploration-exploitation tradeoff — a policy that never explores risks permanently missing the true best action.


```

--- Final Q-values Greedy (n_steps=10000) ---
2 + 3 = ?          -> Q: 0.0000 (Shown 0 times)
14 + 8 = ?         -> Q: 0.0000 (Shown 0 times)
9 - 4 = ?          -> Q: 0.8458 (Shown 10000 times)
17 - 9 = ?         -> Q: 0.0000 (Shown 0 times)
3 x 4 = ?          -> Q: 0.0000 (Shown 0 times)
6 x 7 = ?          -> Q: 0.0000 (Shown 0 times)
9 x 8 = ?          -> Q: 0.0000 (Shown 0 times)
? + 2 = 5          -> Q: 0.0000 (Shown 0 times)
? + 6 = 13         -> Q: 0.0000 (Shown 0 times)
? + 7 = 20         -> Q: 0.0000 (Shown 0 times)

```

```

In [9]: # Part 5: Comparative framework – Greedy agent with 3 initializations
# zero: Q0 = 0.0 | random: Q0 ~ Uniform(0,1) | optimistic: Q0 = 5.

def run_greedy_init(n_steps, q_init):
    ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]

    for op in ops:
        if q_init == 'zero':
            op.q_value = 0.0
        elif q_init == 'random':
            op.q_value = random.random()
        else:
            op.q_value = 5.0 # above any possible reward

    tried_history = []
    q_history = []
    cumreward_curve = []
    total_reward = 0.0

    for t in range(n_steps):
        max_q = max(op.q_value for op in ops)
        best_ops = [op for op in ops if op.q_value == max_q] # collect all best ops
        selected = random.choice(best_ops) # break ties randomly

        reward = selected.show_to_player()
        selected.learn(reward)

        total_reward += reward
        tried_history.append(selected.name)
        q_history.append([op.q_value for op in ops])
        cumreward_curve.append(total_reward)

    return ops, tried_history, np.array(q_history), cumreward_curve

def compute_optimal_pct_init(n_steps, q_init, n_runs=2000):
    optimal_matrix = np.zeros((n_runs, n_steps)) # rows = runs, cols = steps

    for run in range(n_runs):
        ops = [MathOperation(operations[i], real_probabilities[i]) for i in range(len(operations))]
        for op in ops:

```

```

        if q_init == 'zero':
            op.q_value = 0.0
        elif q_init == 'random':
            op.q_value = random.random()
        else:
            op.q_value = 5.0

    for t in range(n_steps):
        max_q = max(op.q_value for op in ops)
        best_ops = [op for op in ops if op.q_value == max_q]
        selected = random.choice(best_ops)
        reward = selected.show_to_player()
        selected.learn(reward)
        if selected.name == operations[optimal_action]:
            optimal_matrix[run, t] = 1

    return np.mean(optimal_matrix, axis=0) * 100

print("Comparative framework functions ready.")

```

Comparative framework functions ready.

```

In [10]: # Comparative plots for 500, 1000, 5000 and 10000 steps
# Each figure overlays the 3 initializations for direct comparison

import plotly.graph_objects as go
from plotly.subplots import make_subplots

INITS = [
    ('zero', 'Q0 = 0', 'crimson'),
    ('random', 'Q0 = random', 'darkorange'),
    ('optimistic', 'Q0 = 5 (optimistic)', '#02C39A'),
]

STEPS_LIST = [500, 1000, 5000, 10000]

for n_steps in STEPS_LIST:

    # run all 3 agents for this step count
    results = {}
    for q_init, label, color in INITS:
        ops, tried_hist, q_arr, cumrew = run_greedy_init(n_steps, q_init)
        pct_opt = compute_optimal_pct_init(n_steps, q_init)
        counts = [tried_hist.count(op) for op in operations]
        results[q_init] = {
            'label': label, 'color': color,
            'ops': ops, 'q_arr': q_arr,
            'cumrew': cumrew, 'pct_opt': pct_opt, 'counts': counts
        }

    steps = list(range(1, n_steps + 1))

```

```

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=[
        '(a) Times each operation was tried',
        '(b) Q-value of optimal action over time',
        '(c) Accumulated reward over time',
        '(d) % Optimal action selected (2000 runs)'
    ]
)

for q_init, label, color in INITS:
    r = results[q_init]

    # (a) grouped bars – one group per operation, one bar per init
    fig.add_trace(go.Bar(
        x=[f'a{i+1}' for i in range(K)],
        y=r['counts'],
        name=label,
        marker_color=color,
        opacity=0.85,
        hovertemplate='<b>' + label + '</b><br>{%x}: {%y} times<ex
    ), row=1, col=1)

    # (b) Q-value of the optimal action only – cleaner comparison
    fig.add_trace(go.Scatter(
        x=steps, y=r['q_arr'][:, optimal_action],
        name=label,
        line=dict(color=color, width=1.5),
        hovertemplate=f'<b>{label}</b><br>Step: {%x}<br>Q optima
    ), row=1, col=2)

    # (c) accumulated reward
    fig.add_trace(go.Scatter(
        x=steps, y=r['cumrew'],
        name=label,
        line=dict(color=color, width=1.5),
        hovertemplate=f'<b>{label}</b><br>Step: {%x}<br>Total co
        showlegend=False
    ), row=2, col=1)

    # (d) % optimal action
    fig.add_trace(go.Scatter(
        x=steps, y=r['pct_opt'],
        name=label,
        line=dict(color=color, width=1.5),
        hovertemplate=f'<b>{label}</b><br>Step: {%x}<br>% Optima
        showlegend=False
    ), row=2, col=2)

    # true q* reference line on (b)
    fig.add_hline(y=real_probabilities[optimal_action], line_dash='dot
        line_color='white', opacity=0.4,

```

```

        annotation_text=f'true q*={real_probabilities[optima
row=1, col=2)

# chance baseline on (d)
fig.add_hline(y=100/K, line_dash='dash', line_color='gray',
              annotation_text=f'Chance ({100/K:.1f}%)', row=2, col=2)

fig.update_layout(
    title=f'Part 5 – Greedy Initialization Comparison | Pedrito Ca
    height=750, width=1100,
    hovermode='x unified',
    barmode='group'
)
fig.update_yaxes(range=[0, 100], row=2, col=2)
fig.show()

from IPython.display import display, Markdown
explanations = {
    500: """
---
**Part 5 – Greedy Initialization Comparison | Steps = 500**

**(a)** The most visible difference appears here.  $Q_0=5$  (optimistic)*
more evenly across all 10 operations because every action starts with
higher than any real reward (0 or 1). The agent must try each action e
its estimate below others, forcing systematic exploration.  $Q_0=0$ ** an
concentrate on fewer actions immediately.

**(b)** The optimistic agent's Q-value for the optimal action drops fr
 $q*=0.90$  as it collects samples.  $Q_0=0$  may have already locked onto the
while  $Q_0=\text{random}$  depends on the initial values drawn.

**(c)** With only 500 steps,  $Q_0=0$  and  $Q_0=\text{random}$  accumulate reward fast
they exploit immediately. The optimistic agent is still in its explorat

**(d)**  $Q_0=5$  may show a lower % optimal action at this step count – it
all actions.  $Q_0=0$  could appear stronger here, but this is due to early
""",
    1000: """
---
**Part 5 – Greedy Initialization Comparison | Steps = 1000**

**(a)** The optimistic agent is starting to concentrate on better acti
downward from 5.0.  $Q_0=0$  remains highly concentrated on its early chose

**(b)** The Q-value curves begin to diverge. The optimistic agent's es
action is now based on more samples and is therefore more reliable tha
few samples if it committed early to a different action.

**(c)** Accumulated reward curves start to differentiate. If  $Q_0=0$  comm
action, its curve will begin to lag behind the optimistic agent which

```

```

**(d)** The optimistic initialization begins to show its advantage: by
early, it has a more accurate picture of which action is truly the best
"""
    5000: """
---
**Part 5 – Greedy Initialization Comparison | Steps = 5000**

**(a)** Clear separation between the three strategies.  $Q_0=5$  has a more
distribution than the others, though it is now converging toward the optimal
 $Q_0=0$  and  $Q_0=\text{random}$  show heavy concentration on one or two actions.

**(b)** All three Q-values are converging toward the true  $q^*=0.90$  (dot
optimistic agent has the most reliable estimate because it collected samples
before committing – reducing the chance of being misled by early lucky
trials.

**(c)** The optimistic agent's accumulated reward curve overtakes  $Q_0=0$ 
phase pays off. The agent that explored more thoroughly now exploits more.

**(d)**  $Q_0=5$  achieves a higher and more stable % optimal action than the others,
confirms the theoretical advantage of optimistic initialization: it trades off
for better long-run convergence to the truly optimal action.
"""
    10000: """
---
**Part 5 – Greedy Initialization Comparison | Steps = 10000**

**(a)** At 10,000 steps, all three initializations have concentrated on the optimal
but  $Q_0=5$  shows the most justified concentration – it earned its commitment.

**(b)** All three Q-value estimates have converged close to  $q^*=0.90$ . However, the
to convergence differed:  $Q_0=5$  explored all options first;  $Q_0=0$  relied on
early lucky trials.

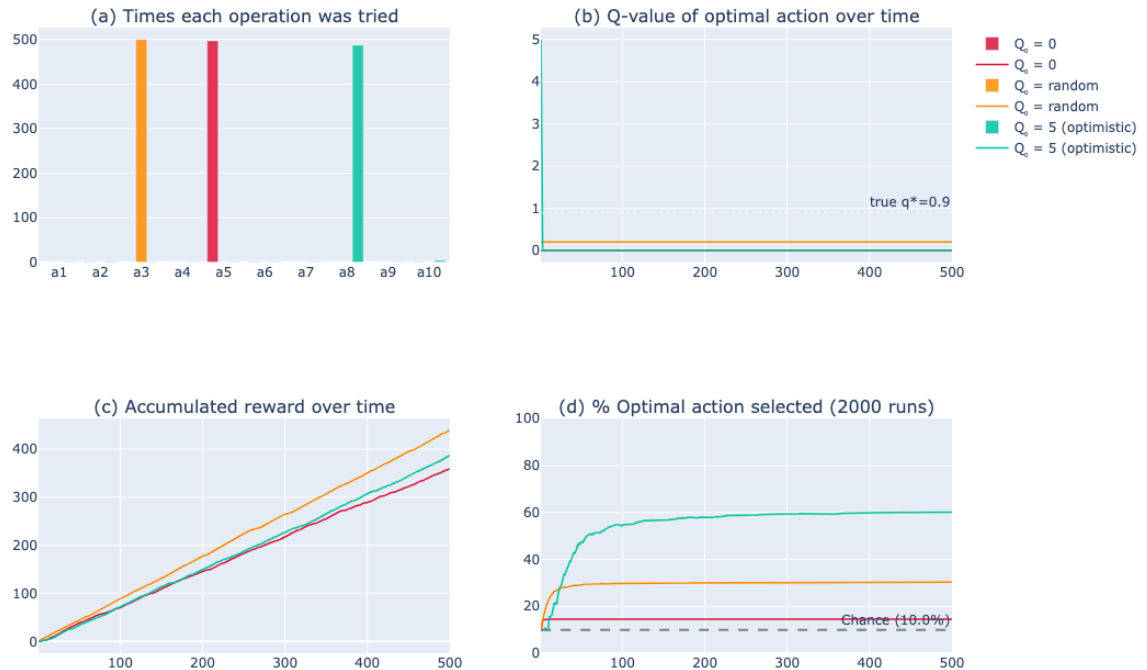
**(c)** The optimistic initialization accumulates the most total reward.
The initial cost of exploration is more than recovered through sustained
exploitation.

**(d)** Final conclusion:  $Q_0=5$  (optimistic) achieves the highest % optimal action,
confirming that initializing above the maximum possible reward forces exploration
systematically before exploiting. This is the key insight of optimistic
exploration for free, without any random policy or  $\epsilon$  parameter.
"""
    }
    display(Markdown(explanations[n_steps]))

    # final Q-values for each initialization
    print(f"\n--- Final Q-values (n_steps={n_steps}) ---")
    for q_init, label, _ in INITS:
        print(f"\n {label}:")
        for op in results[q_init]['ops']:
            print(f"    {op}")

```

Part 5 — Greedy Initialization Comparison | Pedrito Cambolo | Steps = 500



Part 5 — Greedy Initialization Comparison | Steps = 500

(a) The most visible difference appears here. $Q_0=5$ (optimistic) distributes attempts more evenly across all 10 operations because every action starts with a Q-value of 5 — higher than any real reward (0 or 1). The agent must try each action enough times to bring its estimate below others, forcing systematic exploration. $Q_0=0$ and $Q_0=\text{random}$ concentrate on fewer actions immediately.

(b) The optimistic agent's Q-value for the optimal action drops from 5.0 toward the true $q^*=0.90$ as it collects samples. $Q_0=0$ may have already locked onto the optimal if lucky, while $Q_0=\text{random}$ depends on the initial values drawn.

(c) With only 500 steps, $Q_0=0$ and $Q_0=\text{random}$ accumulate reward faster early on because they exploit immediately. The optimistic agent is still in its exploration phase.

(d) $Q_0=5$ may show a lower % optimal action at this step count — it is still exploring all actions. $Q_0=0$ could appear stronger here, but this is due to early commitment, not learning.

--- Final Q-values (n_steps=500) ---

$Q_0 = 0$:

2 + 3 = ?	-> Q: 0.0000	(Shown 0 times)
14 + 8 = ?	-> Q: 0.0000	(Shown 1 times)
9 - 4 = ?	-> Q: 0.0000	(Shown 0 times)
17 - 9 = ?	-> Q: 0.0000	(Shown 0 times)
3 x 4 = ?	-> Q: 0.7223	(Shown 497 times)
6 x 7 = ?	-> Q: 0.0000	(Shown 1 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 0 times)
? + 2 = 5	-> Q: 0.0000	(Shown 0 times)
? + 6 = 13	-> Q: 0.0000	(Shown 0 times)
? + 7 = 20	-> Q: 0.0000	(Shown 1 times)

$Q_0 = \text{random}$:

2 + 3 = ?	-> Q: 0.2062	(Shown 0 times)
14 + 8 = ?	-> Q: 0.4100	(Shown 0 times)
9 - 4 = ?	-> Q: 0.8780	(Shown 500 times)
17 - 9 = ?	-> Q: 0.0269	(Shown 0 times)
3 x 4 = ?	-> Q: 0.4724	(Shown 0 times)
6 x 7 = ?	-> Q: 0.1502	(Shown 0 times)
9 x 8 = ?	-> Q: 0.4660	(Shown 0 times)
? + 2 = 5	-> Q: 0.1110	(Shown 0 times)
? + 6 = 13	-> Q: 0.4802	(Shown 0 times)
? + 7 = 20	-> Q: 0.4393	(Shown 0 times)

$Q_0 = 5$ (optimistic):

2 + 3 = ?	-> Q: 0.0000	(Shown 1 times)
14 + 8 = ?	-> Q: 0.5000	(Shown 2 times)
9 - 4 = ?	-> Q: 0.0000	(Shown 1 times)
17 - 9 = ?	-> Q: 0.0000	(Shown 1 times)
3 x 4 = ?	-> Q: 0.0000	(Shown 1 times)
6 x 7 = ?	-> Q: 0.0000	(Shown 1 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 1 times)
? + 2 = 5	-> Q: 0.7885	(Shown 487 times)
? + 6 = 13	-> Q: 0.0000	(Shown 1 times)
? + 7 = 20	-> Q: 0.5000	(Shown 4 times)

Part 5 — Greedy Initialization Comparison | Pedrito Cambolo | Steps = 1000



Part 5 — Greedy Initialization Comparison | Steps = 1000

(a) The optimistic agent is starting to concentrate on better actions as estimates converge downward from 5.0. $Q_0=0$ remains highly concentrated on its early chosen action.

(b) The Q-value curves begin to diverge. The optimistic agent's estimate of the optimal action is now based on more samples and is therefore more reliable than $Q_0=0$, which may have few samples if it committed early to a different action.

(c) Accumulated reward curves start to differentiate. If $Q_0=0$ committed to a suboptimal action, its curve will begin to lag behind the optimistic agent which has explored more.

(d) The optimistic initialization begins to show its advantage: by forcing exploration early, it has a more accurate picture of which action is truly the best.

--- Final Q-values (n_steps=1000) ---

$Q_0 = 0$:

$2 + 3 = ?$	-> Q: 0.0000	(Shown 0 times)
$14 + 8 = ?$	-> Q: 0.0000	(Shown 0 times)
$9 - 4 = ?$	-> Q: 0.0000	(Shown 0 times)
$17 - 9 = ?$	-> Q: 0.0000	(Shown 0 times)
$3 \times 4 = ?$	-> Q: 0.7710	(Shown 1000 times)
$6 \times 7 = ?$	-> Q: 0.0000	(Shown 0 times)
$9 \times 8 = ?$	-> Q: 0.0000	(Shown 0 times)
$? + 2 = 5$	-> Q: 0.0000	(Shown 0 times)
$? + 6 = 13$	-> Q: 0.0000	(Shown 0 times)
$? + 7 = 20$	-> Q: 0.0000	(Shown 0 times)

$Q_0 = \text{random}$:

$2 + 3 = ?$	-> Q: 0.6364	(Shown 11 times)
$14 + 8 = ?$	-> Q: 0.1356	(Shown 0 times)
$9 - 4 = ?$	-> Q: 0.8521	(Shown 987 times)
$17 - 9 = ?$	-> Q: 0.4287	(Shown 0 times)
$3 \times 4 = ?$	-> Q: 0.1613	(Shown 0 times)
$6 \times 7 = ?$	-> Q: 0.0000	(Shown 1 times)
$9 \times 8 = ?$	-> Q: 0.0000	(Shown 1 times)
$? + 2 = 5$	-> Q: 0.4683	(Shown 0 times)
$? + 6 = 13$	-> Q: 0.1436	(Shown 0 times)
$? + 7 = 20$	-> Q: 0.4899	(Shown 0 times)

$Q_0 = 5$ (optimistic):

$2 + 3 = ?$	-> Q: 0.8936	(Shown 968 times)
$14 + 8 = ?$	-> Q: 0.5000	(Shown 2 times)
$9 - 4 = ?$	-> Q: 0.0000	(Shown 1 times)
$17 - 9 = ?$	-> Q: 0.8333	(Shown 6 times)
$3 \times 4 = ?$	-> Q: 0.0000	(Shown 1 times)
$6 \times 7 = ?$	-> Q: 0.0000	(Shown 1 times)
$9 \times 8 = ?$	-> Q: 0.0000	(Shown 1 times)
$? + 2 = 5$	-> Q: 0.8571	(Shown 14 times)
$? + 6 = 13$	-> Q: 0.8000	(Shown 5 times)
$? + 7 = 20$	-> Q: 0.0000	(Shown 1 times)

Part 5 — Greedy Initialization Comparison | Pedrito Cambolo | Steps = 5000



Part 5 — Greedy Initialization Comparison | Steps = 5000

(a) Clear separation between the three strategies. $Q_0=5$ has a more balanced action distribution than the others, though it is now converging toward the optimal action. $Q_0=0$ and $Q_0=\text{random}$ show heavy concentration on one or two actions.

(b) All three Q-values are converging toward the true $q^*=0.90$ (dotted line), but the optimistic agent has the most reliable estimate because it collected samples from all actions before committing — reducing the chance of being misled by early lucky rewards.

(c) The optimistic agent's accumulated reward curve overtakes $Q_0=0$ as its exploration phase pays off. The agent that explored more thoroughly now exploits more effectively.

(d) $Q_0=5$ achieves a higher and more stable % optimal action than the other two. This confirms the theoretical advantage of optimistic initialization: it trades early performance for better long-run convergence to the truly optimal action.

--- Final Q-values (n_steps=5000) ---

$Q_0 = 0$:

2 + 3 = ?	-> Q: 0.0000	(Shown 0 times)
14 + 8 = ?	-> Q: 0.0000	(Shown 0 times)
9 - 4 = ?	-> Q: 0.0000	(Shown 0 times)
17 - 9 = ?	-> Q: 0.0000	(Shown 0 times)
3 x 4 = ?	-> Q: 0.0000	(Shown 0 times)
6 x 7 = ?	-> Q: 0.0000	(Shown 0 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 0 times)
? + 2 = 5	-> Q: 0.7990	(Shown 5000 times)
? + 6 = 13	-> Q: 0.0000	(Shown 0 times)
? + 7 = 20	-> Q: 0.0000	(Shown 0 times)

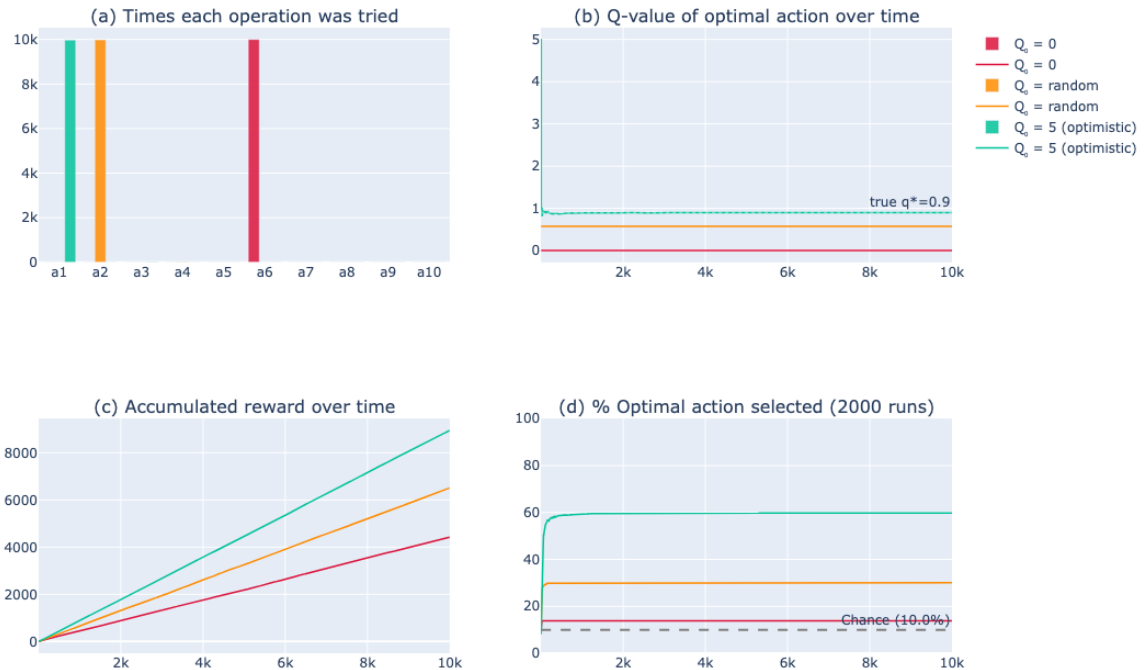
$Q_0 = \text{random}$:

2 + 3 = ?	-> Q: 0.9002	(Shown 5000 times)
14 + 8 = ?	-> Q: 0.4291	(Shown 0 times)
9 - 4 = ?	-> Q: 0.1213	(Shown 0 times)
17 - 9 = ?	-> Q: 0.3814	(Shown 0 times)
3 x 4 = ?	-> Q: 0.5672	(Shown 0 times)
6 x 7 = ?	-> Q: 0.7116	(Shown 0 times)
9 x 8 = ?	-> Q: 0.8497	(Shown 0 times)
? + 2 = 5	-> Q: 0.2409	(Shown 0 times)
? + 6 = 13	-> Q: 0.0567	(Shown 0 times)
? + 7 = 20	-> Q: 0.2346	(Shown 0 times)

$Q_0 = 5$ (optimistic):

2 + 3 = ?	-> Q: 0.8971	(Shown 4964 times)
14 + 8 = ?	-> Q: 0.0000	(Shown 1 times)
9 - 4 = ?	-> Q: 0.7500	(Shown 24 times)
17 - 9 = ?	-> Q: 0.5000	(Shown 2 times)
3 x 4 = ?	-> Q: 0.0000	(Shown 1 times)
6 x 7 = ?	-> Q: 0.0000	(Shown 1 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 1 times)
? + 2 = 5	-> Q: 0.5000	(Shown 2 times)
? + 6 = 13	-> Q: 0.0000	(Shown 1 times)
? + 7 = 20	-> Q: 0.6667	(Shown 3 times)

Part 5 — Greedy Initialization Comparison | Pedrito Cambolo | Steps = 10000



Part 5 — Greedy Initialization Comparison | Steps = 10000

(a) At 10,000 steps, all three initializations have concentrated on their preferred actions, but $Q_0=5$ shows the most justified concentration — it earned its commitment through exploration.

(b) All three Q-value estimates have converged close to $q^*=0.90$. However, the path to convergence differed: $Q_0=5$ explored all options first; $Q_0=0$ relied on luck in early steps.

(c) The optimistic initialization accumulates the most total reward over 10,000 steps. The initial cost of exploration is more than recovered through sustained optimal exploitation.

(d) Final conclusion: $Q_0=5$ (optimistic) achieves the highest % optimal action, confirming that initializing above the maximum possible reward forces the agent to explore systematically before exploiting. This is the key insight of optimistic initialization: **exploration for free, without any random policy or ϵ parameter.**

--- Final Q-values (n_steps=10000) ---

$Q_0 = 0$:

2 + 3 = ?	-> Q: 0.0000	(Shown 0 times)
14 + 8 = ?	-> Q: 0.0000	(Shown 0 times)
9 - 4 = ?	-> Q: 0.0000	(Shown 0 times)
17 - 9 = ?	-> Q: 0.0000	(Shown 0 times)
3 x 4 = ?	-> Q: 0.0000	(Shown 0 times)
6 x 7 = ?	-> Q: 0.4424	(Shown 10000 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 0 times)
? + 2 = 5	-> Q: 0.0000	(Shown 0 times)
? + 6 = 13	-> Q: 0.0000	(Shown 0 times)
? + 7 = 20	-> Q: 0.0000	(Shown 0 times)

$Q_0 = \text{random}$:

2 + 3 = ?	-> Q: 0.5760	(Shown 0 times)
14 + 8 = ?	-> Q: 0.6512	(Shown 9979 times)
9 - 4 = ?	-> Q: 0.4220	(Shown 0 times)
17 - 9 = ?	-> Q: 0.6316	(Shown 19 times)
3 x 4 = ?	-> Q: 0.5042	(Shown 0 times)
6 x 7 = ?	-> Q: 0.0057	(Shown 0 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 1 times)
? + 2 = 5	-> Q: 0.5564	(Shown 0 times)
? + 6 = 13	-> Q: 0.0000	(Shown 1 times)
? + 7 = 20	-> Q: 0.2487	(Shown 0 times)

$Q_0 = 5$ (optimistic):

2 + 3 = ?	-> Q: 0.8956	(Shown 9972 times)
14 + 8 = ?	-> Q: 0.0000	(Shown 1 times)
9 - 4 = ?	-> Q: 0.8000	(Shown 15 times)
17 - 9 = ?	-> Q: 0.0000	(Shown 1 times)
3 x 4 = ?	-> Q: 0.0000	(Shown 1 times)
6 x 7 = ?	-> Q: 0.5000	(Shown 2 times)
9 x 8 = ?	-> Q: 0.0000	(Shown 1 times)
? + 2 = 5	-> Q: 0.0000	(Shown 1 times)
? + 6 = 13	-> Q: 0.8000	(Shown 5 times)
? + 7 = 20	-> Q: 0.0000	(Shown 1 times)